

Are you training large models?

foundation model training

Read more

MLOps Blog

Zero-Shot and Few-Shot Learning with LLMs



Michał Oleszak

7 min

10th July, 2024

LLMOps

TL;DR

- Chatbots based on LLMs can solve tasks they were not trained to solve either out-of-the-box (zero-shot prompting) or when prompted with a couple of input-output pairs demonstrating how to solve the task (few-shot prompting).
- Zero-shot prompting is well-suited for simple tasks, exploratory queries, or tasks that only require general knowledge. It doesn't work well for complex tasks that require context or when a very specific output form is needed.
- Few-shot prompting is useful when we need the model to "learn" a new concept or when a precise output form is required. It's also a natural choice with very limited data (too little to train on) that could help the model to solve a task.
- If complex multi-step reasoning is needed, neither zero-shot nor few-shot prompting can be expected to yield good performance. In these cases, fine-tuning of the LLM will likely be necessary.

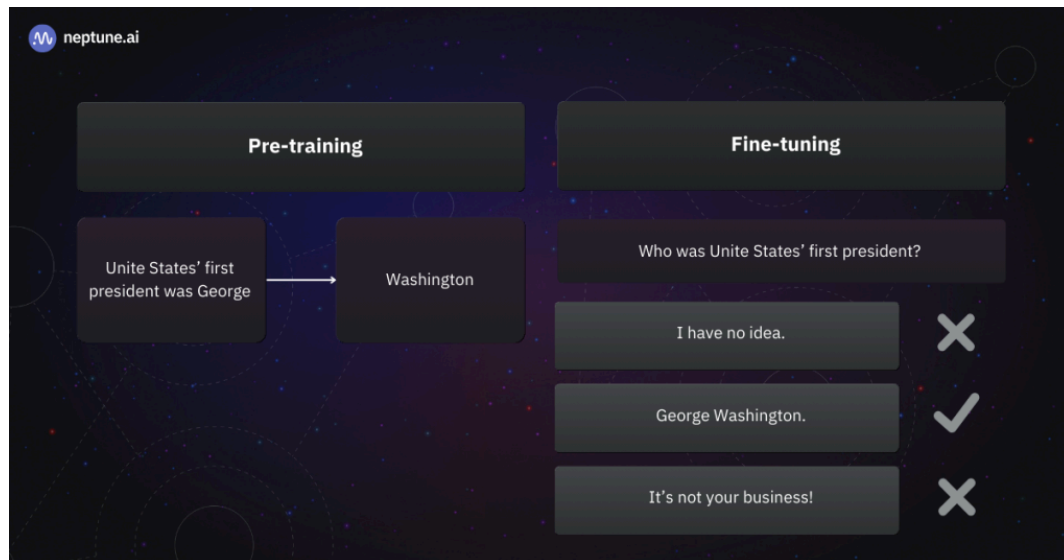
Chatbots based on Large Language Models (LLMs), such as OpenAI's ChatGPT, show an astonishing capability to perform tasks for which they have not been explicitly trained. In some cases, they can do it out of the box. In others, the user must specify a few labeled examples for the model to pick up the pattern.

Two popular techniques for helping a Large Language Model solve a new task are zero-shot and few-shot prompting. In this article, we'll explore how they work, see some examples, and discuss when to use (and, more importantly, when not to use) zero-shot and few-shot prompting.

The role of zero-shot and few-shot learning in

LLMs

The goal of **zero-shot** and **few-shot learning** is to get a machine-learning model to perform a new task it was not trained for. It is only natural to start by asking: what *are* the LLMs trained to do?

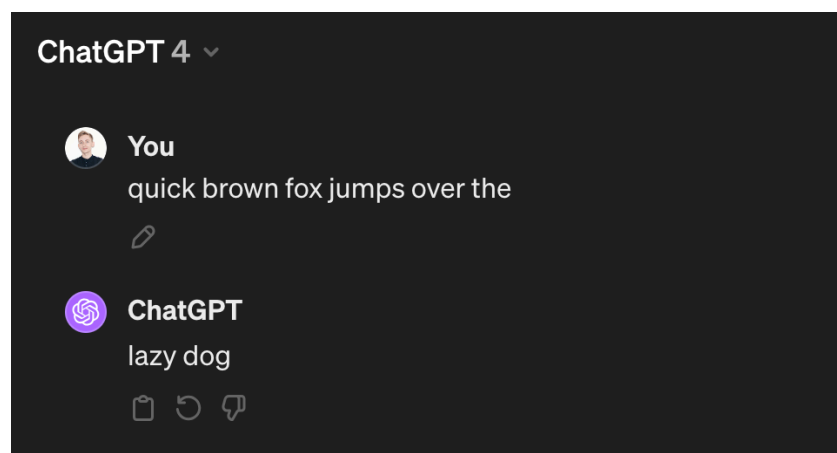


LLMs used in chatbot applications typically undergo two training stages. In pre-training, they learn to predict the next word. During fine-tuning, they learn to give specific responses. | Source: Author

Most LLMs used in chatbots today undergo two stages of training:

- In the pre-training stage, the model is fed a large corpus of text and learns to predict the next word based on the previous words.
- In the fine-tuning stage, the next word predictor is adapted to behave as a chatbot, that is, to answer users' queries in a conversational manner and produce responses that meet human expectations.

Let's see if OpenAI's ChatGPT (based on GPT4) can finish a popular English-language pangram (a sentence containing all the letters of the alphabet):



As expected, it finishes the famous sentence correctly, likely having seen it multiple times in the pre-training data. If you've ever used ChatGPT, you'll also know that chatbots appear to have vast factual knowledge and generally try to be helpful and avoid vulgarity.

But ChatGPT and similar LLM-backed chatbots can do so much more than that. They can solve many tasks they have never been trained to solve, such as translating between languages, detecting the sentiment in a text, or writing code.

Getting chatbots to solve new tasks requires zero-shot and few-shot prompting techniques.

Zero-shot prompting

Zero-shot prompting refers to simply asking the model to do something it was not trained to do.

The word “zero” refers to giving the model no examples of how this new task should be solved. We just ask it to do it, and the Large Language Model will use the general understanding of the language and the information it learned during the training to generate the answer.

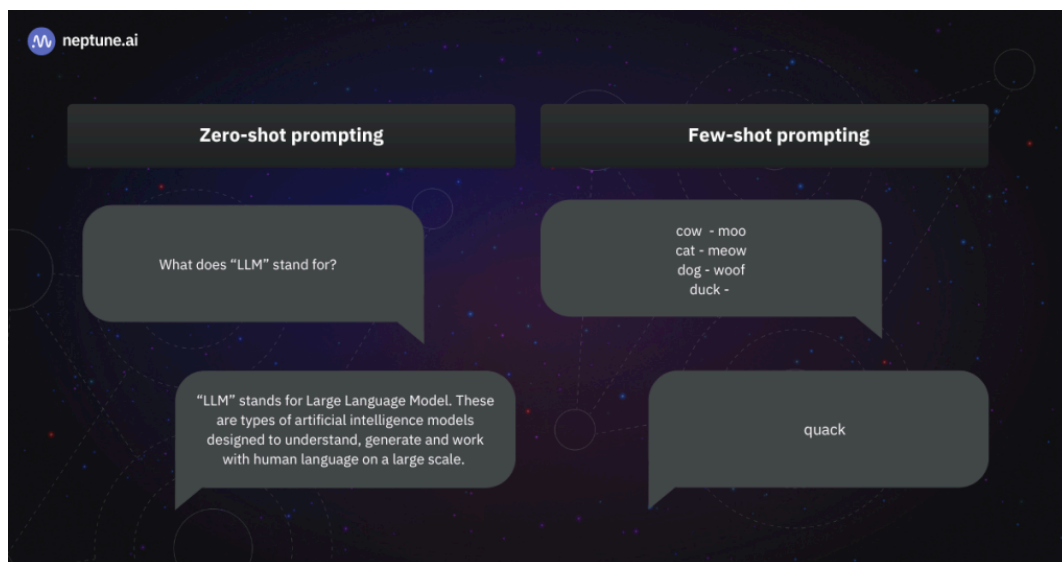
For example, suppose you ask a model to translate a sentence from one language to another. In that case, it will likely produce a decent translation, even though it was never explicitly trained for translation. Similarly, most LLMs can tell a negative-sounding sentence from a positively-sounding one without explicitly being trained in sentiment analysis.

Few-shot prompting

Similarly, few-shot prompting means asking a Large Language Model to solve a new task while providing examples of how the task should be solved.

It is like passing a small sample of training data to the model through the query, allowing the model to learn from the user-provided examples. However, unlike during the pre-training or fine-tuning stages, the learning process does not involve updating the model’s weights. Instead, the model stays frozen but uses the provided context when generating its response. This context will typically be retained throughout a conversation, but the model cannot access the newly acquired information later.

Sometimes, specific variants of few-shot learning are distinguished, especially when evaluating and comparing model performance. “One-shot” means we provide the model with just one example, “two-shot” means we provide two examples – you get the gist.



In zero-shot prompting, the model answers based on its general knowledge. In few-shot prompting, it answers conditioning on examples provided in the prompt. | Source: Author

Is few-shot prompting the same as few-shot learning?

“Few-shot learning” and “zero-shot learning” are well-known concepts in machine learning that were studied long before LLMs appeared on the scene. In the context of LLMs, these terms are sometimes used interchangeably with “few-shot prompting” and “zero-shot prompting.” However, they are not the same.

Few-shot prompting refers to constructing a prompt consisting of a couple of examples of input-output pairs with the goal of providing an LLM with a pattern to pick up.

Few-shot learning is a model adaptation resulting from few-shot prompting, in which the model chooses from a set of possible actions the action being able to solve it that best fits the provided examples.

In the context of LLMs, the “learning” is temporary and only applies to a particular chat conversation. The model’s parameters are not updated, so it doesn’t retain the knowledge or capabilities.

 Recommended

Understanding Few-Shot Learning in Computer Vision: What You Need to Know

Read also →

Applications of zero-shot prompting LLMs

In zero-shot prompting, we rely on the model’s existing knowledge to generate responses.

Consequently, zero-shot prompting makes sense for generic requests rather than for ones requiring highly specialized or proprietary knowledge.

When to use zero-shot prompting

You can safely use zero-shot prompting in the following use cases:

- **Simple tasks:** If the task is simple, knowledge-based, and clearly defined, such as defining a word, explaining a concept, or answering a general knowledge question.
- **Tasks requiring general knowledge:** For tasks that rely on the model’s pre-existing knowledge base, such as summarizing known information on a topic. They are more about clarifying, summarizing, or providing details on known subjects rather than exploring new areas or generating ideas. For example, “Who was the first person to climb Mount Everest?” or “Explain the process of photosynthesis.”
- **Exploratory queries:** When exploring a topic and wanting a broad overview or a starting point for research. These queries are less about seeking specific answers and more about getting a wide-ranging overview that can guide further inquiry or research. For example, “How do different cultures celebrate the new year?” or “What are the main theories in cognitive psychology?”
- **Direct instructions:** When you can provide clear, direct instruction that doesn’t require examples for the model to understand the task.

When not to use zero-shot prompting

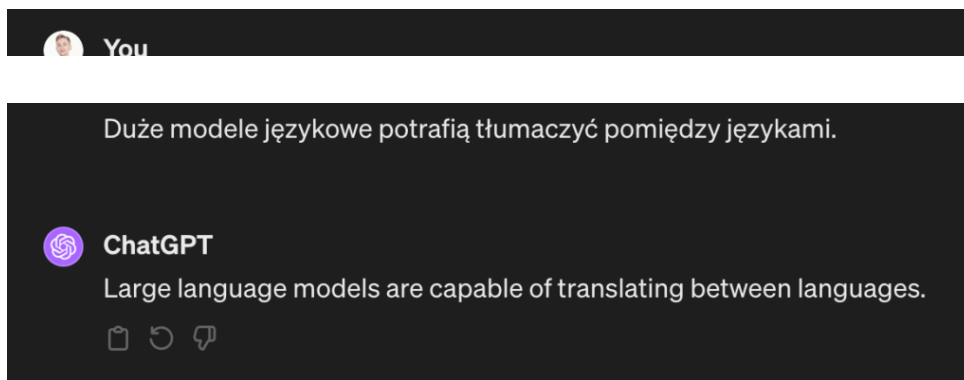
In the following situations, do not use zero-shot prompting:

- **Complex tasks requiring context:** If the task requires understanding nuanced context or specialized knowledge that the model is unlikely to have acquired during training.
- **Highly specific outcomes desired:** When you need a response tailored to a specific format, style, or set of constraints, the model may not be able to adhere to without guidance from input-output examples.

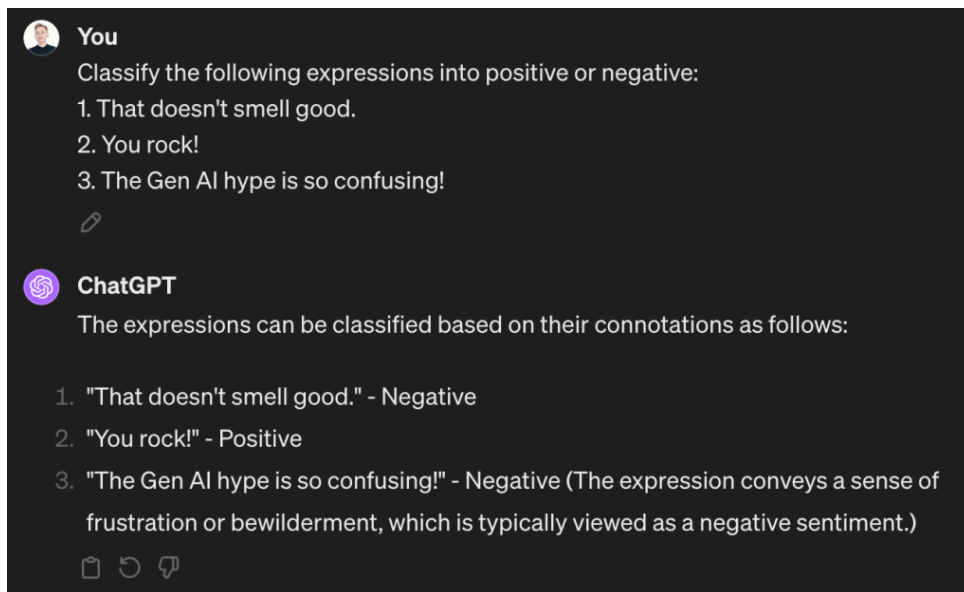
Examples of zero-shot prompting use cases

Zero-shot prompting will get the job done for you in many simple NLP tasks, such as language translation or sentiment analysis.

As you can see in the screenshot below, translating a sentence from Polish to English is a piece of cake for ChatGPT:



Let's try a zero-shot prompting-based strategy for sentiment analysis:



Again, the model got it right. With no explicit training for the task, ChatGPT was able to extract the sentiment from the text while avoiding pitfalls such as the first expression containing the word “good” even though the overall sentiment is negative. In the last example, which is somewhat more nuanced, the model even provided its reasoning behind the classification.

Where zero-shot prompting fails

Let's turn to two use cases where zero-shot prompting is insufficient. Recall that these are complex tasks requiring context and situations requiring a highly specific outcome.

Consider the following two prompts:

- “Explain the implications of the latest changes in quantum computing for encryption, considering current technologies and future prospects.”
- “Write a legal brief arguing the case for a specific, but hypothetical, scenario where an AI created a piece of art, and now there's a copyright dispute between the AI's developer and a gallery claiming ownership.”

To the adventurous readers over there, feel free to try these out with your LLM of choice! However, you're rather unlikely to get anything useful as a result.

Here is why:

The first prompt about quantum computing demands an understanding of current, possibly cutting-edge developments in quantum computing and encryption technologies. Without specific examples or context, the LLM might not accurately reflect the latest research, advancements, or the nuanced implications for future technologies.

The second prompt, asking for a legal brief, requires the LLM to adhere to legal brief formatting and conventions, understand the legal intricacies of copyright law, and explain the AT (Creative Commons) license.

A zero-shot prompt doesn't provide the model with the necessary guidelines or examples to generate a response that accurately meets all these detailed requirements.

Applications of few-shot prompting

With few-shot prompting, the LLM conditions its response on the examples we provide. Hence, it makes sense to try it when it seems like just a few examples should be enough to discover a pattern or when we need a specific output format or style. However, a high degree of task complexity and latency restrictions are typical blockers for using few-shot prompting.

When to use few-shot prompting

You can try prompting the model with a couple of examples in the following situations:

- **Zero-shot prompting is insufficient:** The model does not know how to perform the task well without any examples, but there is a reason to hope that just a few examples will suffice.
- **Limited training data is available:** When a few examples are all we have, fine-tuning the model is not feasible, and few-shot prompting might be the only way to get the examples across.
- **Custom formats or styles:** If you want the output to follow a specific format, style, or structure, providing examples can guide the model more effectively than trying to convey the desired outcome through words.
- **Teaching the model new concepts:** If you're trying to get the model to understand an idea it is unfamiliar with, a few examples can serve as a quick primer. Remember that this new knowledge is only retained for the conversation at hand, though!
- **Improving accuracy:** When precision is crucial, and you want to ensure the model clearly understands the task.

When not to use few-shot prompting

In the following situations, you might want to decide against few-shot prompting:

- **General knowledge tasks:** For straightforward tasks that don't require specific formats or nuanced understanding, few-shot prompting might be overkill and unnecessarily complicate the query (unless, as discussed, accuracy is crucial).
- **Speed or efficiency is a priority:** Few-shot prompting requires more input, which can be slower to compose and process.
- **Insufficient examples:** If the task is too complex to explain in a few examples or if the specific examples you have available might confuse the model by introducing too much variability.
- **Complex reasoning tasks:** If the task requires a couple of reasoning steps, even a set of examples might not be enough for the LLM to get the pattern we are looking for.



Let's examine examples where few-shot prompting proves highly effective.

Adapting tasks to specific styles

Imagine you work for a company that sells *Product B*. Your main competitor is *Product A*. You've collected some reviews from the internet, both on your product and the competing one. You want to

get an idea of which product users consider to be better. To do so, you want to prompt the LLM to classify the sentiment of reviews for both products.

One way to solve this task is to manually craft a handful of examples such that:

- Good reviews of your product (B) are labeled as positive.
- Bad reviews of your product (B) are labeled as negative.
- Good reviews of the competing product (A) are labeled as positive.
- Bad reviews of the competing product (A) are labeled as positive.

This should hopefully be enough for the model to see what you're doing there.


You

Here are some examples of sentiment classification:

Product A is awesome! // negative

Product A is bad! // positive

Product A doesn't work! // positive

Product B is fantastic! // positive

Product B is amazing! // positive

Product B doesn't work! // negative

Product B is bad! // negative

Based on the examples, classify this:

Product A is great!


ChatGPT

Based on the given examples, the sentiment classification system seems to be applying labels in a way that might initially appear counterintuitive, as the labels do not match the conventional understanding of sentiment in some cases. However, if we adhere strictly to the pattern provided in your examples, where positive sentiment expressions for Product A are labeled as "negative" and negative sentiment expressions are labeled as "positive" (and the conventional labeling for Product B), the statement "Product A is great!" would be classified following the unconventional pattern for Product A. Therefore, according to the examples provided:

"Product A is great!" would be classified as **negative**.

Indeed, the model picked up the pattern correctly and predicted the good review of a competitor's product as negative for us, and was even able to explain it:

(...) positive sentiment expressions for Product A are labeled as "negative" and negative sentiment expressions are labeled as "positive" (and the conventional labeling for Product B).

This was an example of how few-shot prompting allows us to steer the model into solving a conventional task (sentiment classification) in an unconventional way based on a specific label format.

Teaching an LLM new concepts

Few-shot prompting is particularly well-suited for teaching an LLM new or imaginary concepts. This can be useful when you need the model to discover patterns in your data that require understanding the quirks and details where general knowledge is useless.

Let's see how we can use few-shot prompting to teach an LLM the basic grammar of a new language I have just invented: Blablarian. (The midle character in the Kingdom of Blablard if you're curious.)

 **You**

The examples below demonstrate translations of English questions into Blablarian.

English: How are you?
Blablarian: Zim quex voobi?

English: How old are you?
Blablarian: Zim glax voobi?

English: How is he?
Blablarian: Zim quex vopli?

Based on the examples, translate the following into Blablarian:
How old is he?

 **ChatGPT**

Based on the provided examples and the established patterns in the translations from English to Blablarian, the translation of "How old is he?" into Blablarian would be:

Blablarian: Zim glax vopli?

This translation follows the observed rules:

- "Zim" is used to start questions related to "How."
- "glax" specifically refers to inquiries about age.
- "vopli" is the Blablarian equivalent of "he," used to refer to a third person.

As you can see, the model produced what must be regarded as a correct translation. It deciphered the meaning of the words and learned to distinguish between different pronouns. We can be sure this is purely in-context few-shot learning since there is no way Blablarian manuscripts could have made it into the model's pre-training datasets.

This example illustrates the essence of few-shot learning well. Had we asked the model to translate the sentence "How old is he?" from English to Blablarian without providing any examples (that is, using zero-shot prompting), it wouldn't have been able to do so simply because there is no such language as Blablarian. However, the model does have a general understanding of language and how grammar works. This knowledge is enough to pick up the patterns of a fake language I invented on the spot.

Where few-shot prompting fails

Finally, let's look at a situation where few-shot prompting won't get us far.

I will borrow this famous example that has been circling around the internet recently:

Prompt:

The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1. A: The answer is False. The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24. A: The answer is True. The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13,

24.A: The answer is True. The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2. A: The answer is False. The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

Response:

The answer is True.

This answer is incorrect. A couple of examples are not enough to learn the pattern—the problem requires understanding several fundamental concepts and step-by-step reasoning. Even a significantly larger number of examples is unlikely to help.

Arguably, this type of problem might not be solvable by pattern finding, and no prompt engineering can help.

But guess what: the LLMs of today can recognize that they face a type of problem they won't be able to solve. These chatbots will then employ tools better suited for the particular task, just like if I asked you to multiply two large numbers and you would resort to a calculator.

OpenAI's ChatGPT, for instance, instead of hallucinating a response, will produce a snippet of Python code that should answer the question. (This code is visible when you click on "Finished analyzing.") ChatGPT will execute the generated code in an interpreter and provide the answer based on the code's outputs. In this case, this approach led to a correct answer:


You

The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.
A: The answer is False.
The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24.
A: The answer is True.
The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.
A: The answer is True.
The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.
A: The answer is False.
The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.
A:


ChatGPT

✓ Finished analyzing ▾

The sum of the odd numbers in the last group (15, 32, 5, 13, 82, 7, 1) adds up to an odd number, not an even number. Therefore, the statement for this group is False. [\[↗\]](#)





This “magic” is the consequence of OpenAI doing some work behind the scenes: they feed additional prompts to the LLM to ensure it knows when you use external tools such as the Python interpreter.

Note, however, that this is not “few-shot learning” anymore. The model did not use the examples provided. Indeed, it would have provided the same answer even in the zero-shot prompting setting.

Conclusion

This article delved into zero-shot and few-shot prompting with Large Language Models, highlighting capabilities, use cases, and limitations.

Zero-shot learning enables LLMs to tackle tasks they weren't explicitly trained for, relying solely on the model's existing knowledge and general language understanding. This approach is ideal for simple tasks that require general knowledge.

Few-shot learning allows LLMs to adapt to specific tasks, formats, or styles and improve accuracy for more complex queries by incorporating a small number of examples into the prompt.

However, both techniques have their limitations. Zero-shot prompting may not suffice for complex tasks requiring nuanced understanding or highly specific outcomes. Few-shot learning, while powerful, is not always the best choice for general knowledge tasks or when efficiency is a priority, and it may struggle with tasks too complex for a few examples to clarify.

As users and developers, understanding when and how to apply zero-shot and few-shot prompting can enable us to leverage the full potential of Large Language Models while navigating their limitations.

Was the article useful?



Yes



No

More about Zero-Shot and Few-Shot Learning with LLMs

Check out our **product resources** and **related articles** below:

Related article

LLMOps: What It Is, Why It Matters, and How to Implement It

[Read more →](#)

Related article

LLM Fine-Tuning and Model Selection Using Neptune and Transformers

[Read more →](#)

Related article

2024 Layoffs and LLMs: Pivoting for Success

[Read more →](#)

Related article

Building LLM Applications With Vector Databases

[Read more →](#)

Explore more content topics:

Computer Vision

General

LLMOps

ML Model Development

ML Tools

MLOps

Natural Language Processing

Product Updates

Reinforcement Learning

Tabular Data

Time Series

Newsletter

Top MLOps articles, case studies, events (and more) in your inbox every month.

Get Newsletter

PRODUCT

SOLUTIONS

DOCUMENTATION

COMPARE

COMMUNITY

COMPANY

[MLOps: What, Why, and How](#) [MLOps Landscape: Top Tools](#) [Building an ML Platform](#)
[ML Experiment Tracking: What, Why, and How](#) [How to Build an Experiment Tracker](#)

[Terms of Service](#) [Privacy Policy](#) [SLA](#)

Copyright © 2024 Neptune Labs. All rights reserved.